

CRM Análisis Tarifas:Análisis de Rentabilidad y Segmentación de Clientes por Tarifa en un Ecommerce Simulado

Trabajo Fin de Máster - Módulo de Python

Alumno: Adrián Soler Carrera

Correo: adrisolercolin@alumnos.cei.es

Máster: Big Data y Python

Centro: CEI Escuela de Diseño y Tecnología

Fecha de entrega: [25/04/2025]



Índice

1. **Introducción y Objetivos**
 - 1.1 Propósito del proyecto
 - 1.2 Objetivo general
 - 1.3 Objetivos específicos
2. **Diseño y Desarrollo**
 - 2.1 Arquitectura general del software
 - 2.2 Decisiones de diseño importantes
 - 2.3 Enfoque de desarrollo
3. **Código Fuente**
 - 3.1 Estructura modular y organización
 - 3.2 Flujo de ejecución del código
4. **Resultados y Pruebas**
 - 4.1 Resultados del procesamiento de datos
 - 4.2 Visualizaciones generadas
 - 4.3 Ejecución en Google Colab
5. **Problemas y Soluciones**
 - 5.1 Estructuración modular del proyecto
 - 5.2 Rutas de archivos y compatibilidad con Google Colab
 - 5.3 Validación de datos y control de errores
 - 5.4 Simulación realista de tarifas
 - 5.5 Problemas de instalación del paquete
6. **Conclusiones y Trabajo Futuro**
 - 6.1 Resumen general del proyecto
 - 6.2 Aprendizajes y logros personales
 - 6.3 Valor del proyecto (aplicabilidad)
 - 6.4 Limitaciones encontradas
 - 6.5 Líneas futuras de mejora
7. **Referencias**
 - 7.1 Documentación técnica y oficial
 - 7.2 Documentación académica
 - 7.3 Herramientas de soporte
 - 7.4 Fuente de datos utilizada
8. **Apéndices**
 - 8.A Diagrama UML del sistema
 - 8.B Comando de empaquetado

1. Introducción y Objetivos

Propósito del proyecto

En la era digital, el comercio electrónico ha adquirido una relevancia significativa en la forma en que las personas compran productos y servicios. Este crecimiento ha motivado a las empresas a implementar estrategias de fidelización, entre ellas los modelos de suscripción. Para llevar a cabo estas estrategias, se requiere de un análisis detallado del comportamiento de los usuarios, la rentabilidad de cada segmento y el impacto real de las tarifas aplicadas.

Este Proyecto de Fin de Máster tiene como objetivo principal el desarrollo de un sistema de análisis de rentabilidad y segmentación de clientes para un ecommerce simulado, implementando una estructura de tarifas tipo "basic" y "premium".

Objetivo general

- Desarrollar un sistema de análisis y segmentación de clientes por tarifa en un ecommerce simulado con suscripción, usando Python como herramienta principal.

Objetivos específicos

- Cargar y combinar diferentes fuentes de datos para obtener un dataset integrado.
- Aplicar técnicas de limpieza, transformación y validación de datos.
- Simular la asignación de tarifas "basic" y "premium" a los clientes.
- Clasificar los productos en categorías principales.
- Generar visualizaciones que permitan analizar el comportamiento y la rentabilidad por cliente, tarifa y categoría de producto.
- Empaquetar el proyecto como un módulo Python reutilizable y compatible con Google Colab.

2. Diseño y Desarrollo

Arquitectura general del software

El sistema ha sido diseñado como un paquete Python completamente modular llamado CRM_Analisis. Este paquete contiene varios módulos, cada uno con responsabilidades claramente definidas:

- **data_loader.py**: Carga y fusiona los datos desde archivos CSV.
- **processor.py**: Limpia, transforma y valida el dataset.

- **customer_analysis.py**: Simula la asignación de tarifas a clientes.
- **visualizer.py**: Genera visualizaciones del comportamiento y rentabilidad de los clientes.
- **exceptions.py**: Define excepciones personalizadas.
- **main.py**: Orquesta toda la ejecución del proyecto.

Decisiones de diseño importantes

- Uso de **Programación Orientada a Objetos (POO)** para encapsular responsabilidades en clases.
- Inclusión de **excepciones personalizadas** para mejorar la robustez del código.
- Uso de **PEP8** y convenciones de nombres estándar.
- Separación de archivos de entrada, salida de datos y gráficos en carpetas dedicadas dentro de `outputs/`.
- Inclusión de un **patrón de diseño simple**, como composición y modularidad para escalabilidad futura.

Decisiones de diseño

- Separación de responsabilidades por módulo.
 - Inclusión de validaciones personalizadas.
 - Compatibilidad con Google Colab y empaquetado en formato `.whl`.
-

3. Código Fuente

El proyecto ha sido desarrollado completamente en Python, organizando la lógica en un paquete modular denominado `CRM_Analisis`. Este enfoque favorece la reutilización del código, su escalabilidad y la claridad en la estructura general del sistema. La modularización ha seguido los principios de la programación orientada a objetos, separando responsabilidades específicas en distintos módulos.

Cada módulo ha sido diseñado con funciones claramente definidas, utilizando clases, excepciones personalizadas y métodos para encapsular la lógica. El flujo general de trabajo abarca desde la carga de datos brutos hasta la generación de visualizaciones interpretables para el análisis estratégico del ecommerce simulado.

Flujo de ejecución del código

1. **Carga de datos**: El módulo `data_loader.py` fusiona los datasets en un solo archivo `crm_data.csv`, unificando la información de pedidos, productos y clientes.

2. **Procesamiento:** `processor.py` limpia y transforma los datos. Calcula el `total_paid`, elimina registros inconsistentes, y clasifica los productos en categorías principales.
3. **Simulación de tarifas:** Con `customer_analysis.py`, se asignan aleatoriamente tarifas "basic" o "premium" a los clientes para simular un modelo de suscripción.
4. **Visualización:** `visualizer.py` genera gráficos que permiten explorar el comportamiento de los clientes, la rentabilidad por categoría, la evolución mensual de ingresos, y el ARPU, entre otros.
5. **main.py:** Actúa como punto de entrada al proyecto, integrando todos los módulos en un flujo automatizado que produce el análisis final y los gráficos

Enfoque de desarrollo

Se ha seguido un enfoque iterativo, validando cada paso mediante ejecución en local y pruebas en Google Colab. El código ha sido documentado siguiendo el estilo Numpy y se han respetado las convenciones de estilo PEP8.

Este diseño estructurado facilita la mantenibilidad y extensibilidad del código, permitiendo futuras mejoras como la incorporación de modelos predictivos o nuevas visualizaciones.

4. Resultados y Pruebas

Una vez completada la arquitectura modular y ejecutado el flujo principal del sistema desde `main.py`, se procedió a validar la funcionalidad de cada uno de los módulos del paquete `CRM_Analisis`, analizando los resultados generados en cada etapa del proceso. Las pruebas realizadas permiten verificar el correcto procesamiento, segmentación y visualización de los datos, alineados con los objetivos propuestos.

4.1 Resultados del procesamiento de datos

El sistema generó correctamente tres archivos principales durante la ejecución:

- `outputs/data/crm_data.csv`: Contiene la fusión completa de los datos originales provenientes de los archivos `customers.csv`, `orders.csv`, `order_items.csv` y `products.csv`.
- `outputs/csv/crm_data_clean.csv`: Versión limpia y enriquecida del dataset, tras aplicar validaciones, transformación de fechas, eliminación de duplicados y creación de la variable `total_paid`.
- `outputs/data/crm_data_con_tarifas.csv`: Archivo final que incluye la asignación aleatoria de tarifas "basic" y "premium" a los clientes.

Se validaron también casos extremos como:

- Filas con valores nulos.
- Clientes con identificadores no válidos.
- Registros con `total_paid` igual o inferior a cero.

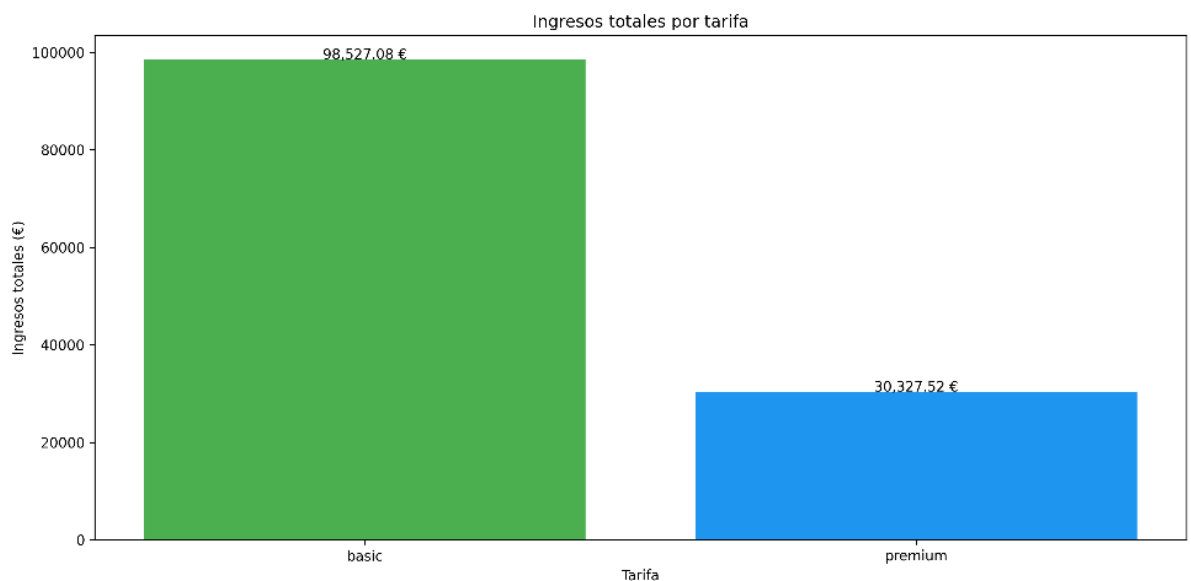
El sistema respondió correctamente, lanzando excepciones personalizadas (`ClienteInvalidoError`, `PrecioInvalidoError`) y deteniendo la ejecución cuando correspondía.

4.2 Visualizaciones generadas

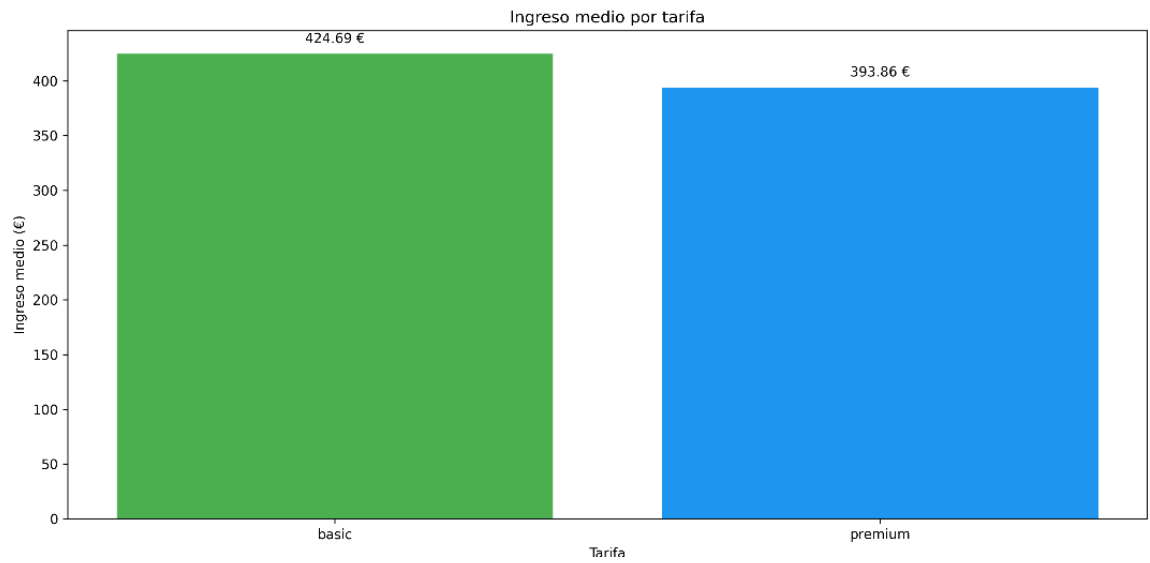
La ejecución del módulo `visualizer.py` generó múltiples gráficos almacenados en `outputs/plots`. Cada uno de ellos está diseñado para analizar un aspecto específico del comportamiento de los clientes y la rentabilidad del modelo de tarifas.

Gráficos destacados:

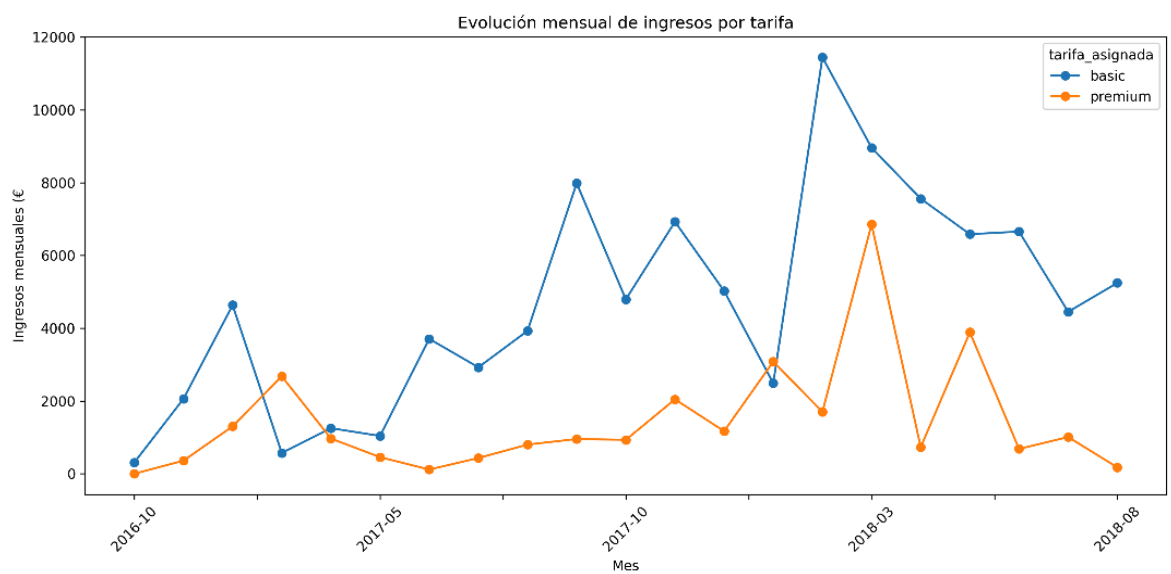
Ingresos totales por tarifa: Permite identificar qué tipo de tarifa genera mayor volumen de ingresos en conjunto.



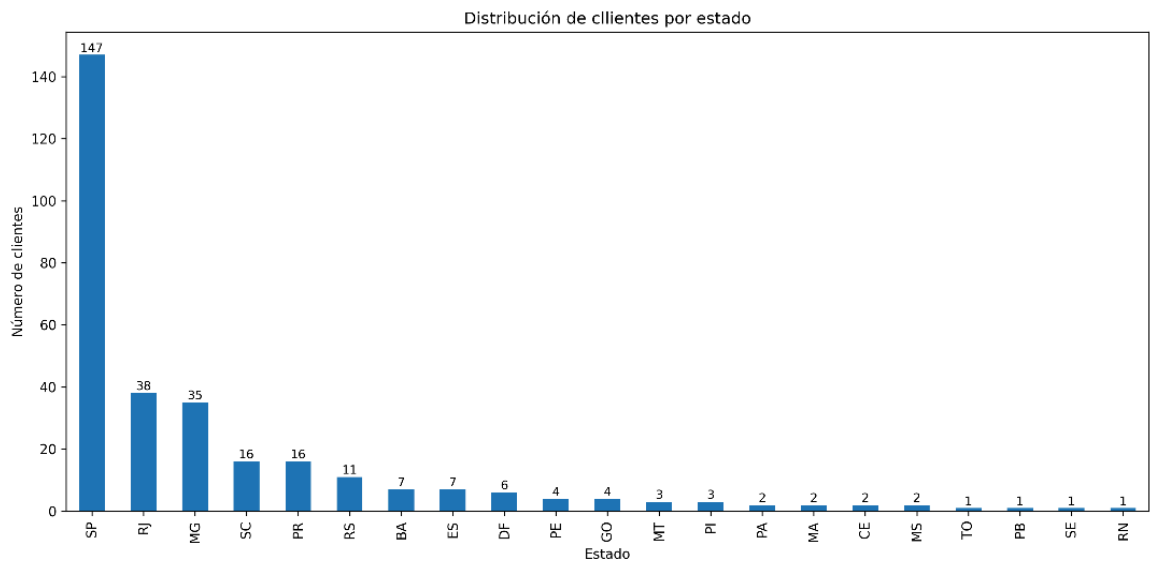
Ingreso medio por tarifa: Muestra el ticket promedio, útil para entender el valor individual de cada cliente.



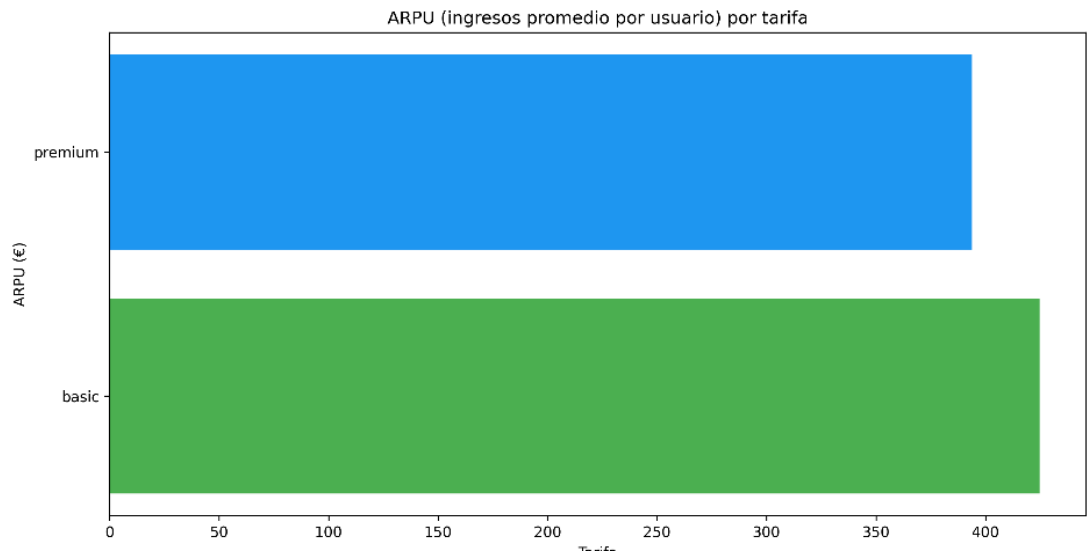
Evolución mensual de ingresos por tarifa: Analiza la tendencia mensual de ingresos, ayudando a detectar patrones estacionales o anomalías.



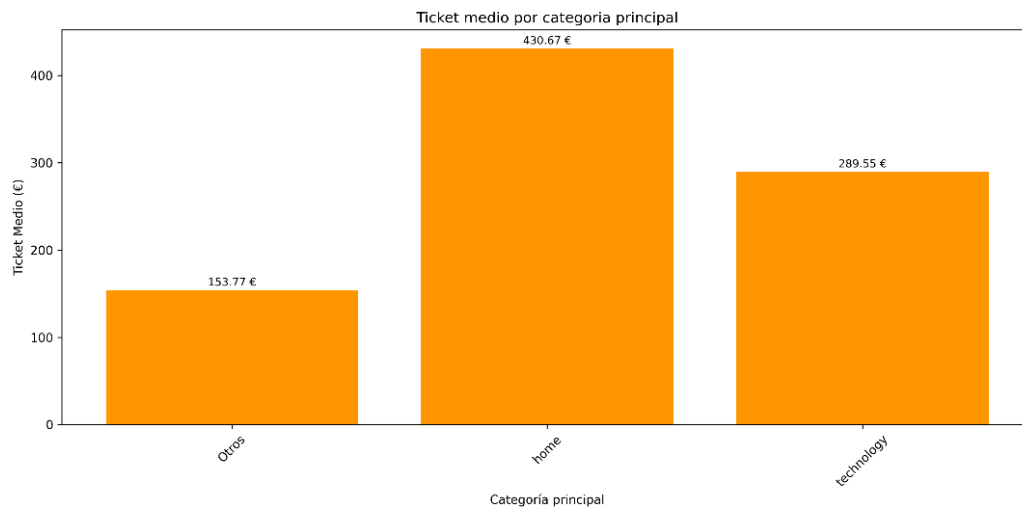
Distribución de clientes por estado: Muestra en qué regiones hay mayor concentración de clientes, lo cual puede ser clave para campañas geográficas.



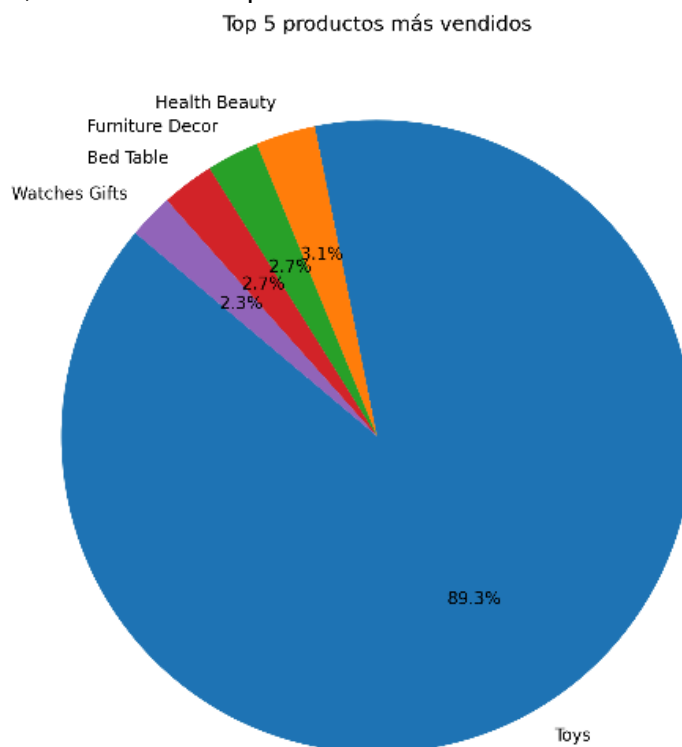
ARPU por tarifa (Average Revenue Per User): Calculado y visualizado para comparar el valor generado por cada cliente en función de su tarifa.



Ticket medio por categoría: Indica qué tipo de productos tienden a ser más caros, con utilidad para decisiones de catálogo o pricing.



Top 5 productos más vendidos (por categoría): Se presenta en un gráfico de tarta, resumiendo las preferencias del consumidor.



Cada gráfico fue diseñado con etiquetas, colores distintivos para cada tarifa, y guardado con calidad profesional (dpi=300) para su posible inclusión en informes o presentaciones.

Conclusiones después del estudio

Tras la exploración de los resultados y las visualizaciones generadas, se extraen las siguientes conclusiones relevantes sobre la rentabilidad y el comportamiento de los clientes en el ecommerce simulado:

- **Ingresos Totales y Medios:**

La tarifa *basic* concentra la mayor parte de los ingresos totales, superando a *premium* tanto en facturación global como en el número de clientes. Sin embargo, el ingreso medio por cliente en *premium* es aproximadamente 30€ superior, lo que indica que, aunque son menos, estos clientes realizan compras de mayor valor.

- **Evolución Temporal:**

El análisis mensual muestra que los picos de ventas se producen en los meses de Navidad de 2017, reflejando un comportamiento estacional típico en ecommerce. La tarifa *basic* domina casi todos los meses en ingresos, salvo una excepción significativa en las Navidades de 2016, donde *premium* tuvo un repunte destacado.

- **Distribución Geográfica:**

El estado con mayores ventas es SP (São Paulo), consolidándose como el principal mercado dentro del dataset. Esto sugiere que las acciones comerciales o de marketing podrían concentrarse en esta región para maximizar resultados.

- **Rentabilidad por Tarifa (ARPU):**

El ARPU (ingreso promedio por usuario) es mayor para los clientes *basic*, a pesar de que la tarifa *premium* tiene un mayor ingreso medio por cliente. Esto puede deberse a la mayor cantidad de usuarios en el segmento *basic*, lo que mejora la rentabilidad agregada.

- **Ticket Medio por Categoría:**

La categoría *home* destaca significativamente, con un ticket medio alrededor de 140 € mayor que el resto de categorías, lo que indica que los productos del hogar generan compras de mayor valor individual.

- **Productos Más Vendidos:**

El análisis del top 5 de productos revela que los juguetes son los artículos más demandados, seguidos por otras categorías menos representadas en las ventas totales.

- **Oportunidades de Mejora:**

Estos resultados permiten concluir que existen oportunidades para reforzar campañas dirigidas tanto a clientes *premium* (por su ticket medio elevado) como para fomentar el cross-selling en la categoría *home*. Igualmente, los patrones estacionales invitan a preparar acciones específicas para los periodos de Navidad.

4.3 Ejecución en Google Colab

El proyecto fue probado con éxito en el entorno de Google Colab para garantizar su portabilidad. Se empaquetó como `.whl` y se instaló con el comando:

```
!pip install /content/crm_analisis_tarifas-1.0.0-py3-none-any.whl
```

Posteriormente, se ejecutaron los módulos directamente desde Colab, replicando toda la funcionalidad del entorno local. Se validó que los gráficos y archivos fueran generados en la nube de forma correcta.

5. Problemas y Soluciones

5.1 Estructuración modular del proyecto

Problema: Al principio del desarrollo, mantener una estructura modular clara y escalable resultó complejo. Al dividir el proyecto en distintos archivos y subcarpetas, surgieron errores de importación, especialmente al trabajar con rutas relativas.

Solución: Se estableció una estructura de paquete en forma de módulo Python (CRM_Análisis) con un archivo `__init__.py` en cada carpeta. Además, se usó `os.path` para gestionar rutas de forma dinámica, asegurando que el código fuera compatible con distintos entornos (PyCharm y Colab).

5.2 Rutas de archivos y compatibilidad con Google Colab

Problema: Al ejecutar el proyecto en Google Colab, muchas rutas de archivos generaban errores ya que Colab utiliza una estructura diferente a la local. Además, los gráficos y archivos `.csv` quedaban desorganizados.

Solución: Se reorganizaron las salidas en carpetas específicas (`outputs/csv/` y `outputs/graficos/`). También se actualizó el código para crear esas carpetas automáticamente con `os.makedirs(..., exist_ok=True)` y se documentó la ruta de los archivos dentro del entorno Colab.

5.3 Validación de datos y control de errores

Problema: Algunos pedidos tienen precios iguales o inferiores a cero, lo cual afectaba el análisis. Además, algunos IDs de cliente no cumplían con el formato esperado.

Solución: Se crearon **excepciones personalizadas** (`PrecioInvalidoError` y `ClienteInvalidoError`) y se implementó su gestión dentro del módulo `processor.py`. Esto permitió detener el procesamiento en caso de errores críticos y facilitar el diagnóstico.

5.4 Simulación realista de tarifas

Problema: No se contaba con una columna real de tarifas, lo que dificultaba la validación de hipótesis y el análisis de evolución por cliente.

Solución: Se simuló una asignación aleatoria de tarifas (25% "premium", 75% "basic") con semilla fija (`random_state=42`) para garantizar la reproducibilidad del análisis. Esto permitió crear un escenario lo más cercano posible a un modelo de negocio real.

5.5 Problemas de instalación del paquete

Problema: Al intentar instalar el paquete .whl en Google Colab, surgieron conflictos con versiones de pandas y errores por rutas de instalación incorrectas.

Solución: Se recompiló el paquete asegurando compatibilidad con pandas==2.2.2, versión requerida por Colab. Además, se ajustaron los comandos de instalación rehaciendo el empaquetado hasta que se optó por iniciar un empaquetamiento nuevo sin usar específicamente el descargado al linio del classroom “cei_template”, ya con uno nuevo que mantenía la estructura se consiguió volver a empaquetar y se probó con éxito en colab.

6. Conclusiones y Trabajo Futuro

6.1 Resumen general del proyecto

El presente proyecto ha logrado desarrollar de manera exitosa un sistema modular de análisis de rentabilidad y segmentación de clientes para un ecommerce simulado, orientado a contextos de suscripción. Se ha implementado una estructura clara en Python que permite cargar, limpiar, transformar, segmentar y visualizar los datos de forma ordenada y replicable.

Este análisis permite evaluar el impacto de las tarifas aplicadas, los productos más rentables y el comportamiento del consumidor, aportando una base sólida para tomar decisiones estratégicas. Desde una perspectiva profesional en CRM, el sistema proporciona una visión integral que podría ser utilizada en entornos reales.

6.2 Aprendizajes y logros personales

A lo largo del desarrollo de este trabajo, se han reforzado de forma significativa competencias en manipulación avanzada de datos con Pandas, programación orientada a objetos (POO), diseño modular de proyectos y generación de visualizaciones efectivas mediante Matplotlib. Asimismo, se ha adquirido experiencia práctica en el empaquetado y distribución de código como un módulo reutilizable, compatible incluso con entornos como Google Colab.

Más allá de las habilidades técnicas, el proyecto ha permitido comprender en profundidad conceptos clave del análisis de clientes, como el ARPU, el ticket medio o el análisis de rentabilidad por categoría y tarifa, todos ellos fundamentales en el ámbito profesional del CRM Analysis.

6.3 Valor del proyecto (aplicabilidad)

El valor práctico de este proyecto es evidente: puede servir como base para el desarrollo de herramientas reales de análisis en empresas que operan bajo modelos de suscripción o requieren segmentar a sus clientes para optimizar sus ingresos. La lógica implementada, aunque simulada, replica fielmente procesos que muchas compañías ejecutan en su día a día.

Este tipo de análisis permitiría a los departamentos de marketing, CRM o dirección comercial tomar decisiones más estratégicas: ajustar tarifas, diseñar campañas personalizadas, mejorar la retención o incluso redefinir el modelo de negocio hacia uno basado en datos.

6.4 Limitaciones encontradas

Una de las principales limitaciones encontradas ha sido la ausencia de un histórico real de tarifas, lo que impide analizar la evolución de los clientes frente a cambios reales de modelo de suscripción. La simulación de tarifas ha permitido avanzar en el análisis, pero carece de contexto temporal o de comportamiento anterior.

Por otro lado, la base de datos utilizada contenía un único pedido por cliente, lo cual ha limitado la posibilidad de realizar un análisis evolutivo personalizado por cliente a lo largo del tiempo. Esto ha restringido el alcance en aspectos como la fidelidad, el ciclo de vida o la predicción de churn.

6.5 Líneas futuras de mejora

Como continuación natural del proyecto, se propone la implementación de un sistema de alertas para detectar clientes en riesgo de abandono, mediante técnicas de scoring o clustering. Además, sería de gran utilidad incorporar modelos de clasificación para predecir qué tipo de tarifa se ajusta mejor a cada cliente, según su comportamiento y rentabilidad esperada.

Otra evolución valiosa sería trabajar con datasets que incluyan compras recurrentes o en distintos periodos temporales, para poder realizar análisis comparativos año a año. Esto permitiría, por ejemplo, identificar clientes que deberían migrar de tarifa, detectar patrones de evolución o rediseñar completamente la estructura de precios y beneficios asociada a cada segmento.

7. Referencias

A continuación, se listan las fuentes y recursos utilizados durante el desarrollo del proyecto, tanto a nivel técnico como académico:

7.1 Documentación oficial y recursos técnicos

- **Pandas:** <https://pandas.pydata.org/docs/>

- **Matplotlib:** <https://matplotlib.org/stable/index.html>
- **NumPy:** <https://numpy.org/doc/>
- **PEP8 - Style Guide for Python Code:** <https://peps.python.org/pep-0008/>
- **Setuptools (Empaquetado de Python):**
<https://setuptools.pypa.io/en/latest/userguide/>

7.2 Documentación académica proporcionada

- **Manual oficial de la asignatura del módulo de Python - CEI Escuela de Diseño y Tecnología:**
Disponible en la plataforma de Google Classroom del máster:
<https://classroom.google.com/w/Njg3ODQxNDg4MjUx/t/all>

Este material ha servido como guía estructural para el diseño del paquete, la construcción de módulos, el uso de excepciones, el empaquetado con `setuptools`, la distribución para Colab y la generación de README y CHANGELOG.

7.3 Herramientas de soporte utilizadas

- **ChatGPT (GPT-4, OpenAI):**
Herramienta utilizada como asistente de programación y documentación para:
 - Generación de estructura modular.
 - Redacción de docstrings y documentación técnica.
 - Diseño de prompts y ayuda en expresiones regulares.
 - Revisión de errores y sugerencias de código.
 - Desarrollo del README.md, setup.py y documentación auxiliar.
- Los prompts utilizados incluyeron descripciones técnicas, errores encontrados, objetivos de análisis o formato de gráficos, y fueron siempre validados y adaptados antes de su implementación directa.

7.4 Fuente de datos utilizada

Dataset de ejemplo para e-commerce simulado

Fuente del dataset:

Los archivos CSV empleados en el proyecto provienen del repositorio público de GitHub:

- [Ecommerce Data Work - Products-Clean.csv](#)

El dataset fue sugerido y adaptado con ayuda de ChatGPT (OpenAI) y se utiliza exclusivamente con fines académicos y de desarrollo.

8. Apéndices

8A: Diagrama UML

8.A.1 Diagrama UML del sistema

Para representar gráficamente la arquitectura del sistema y las relaciones entre los módulos desarrollados, se ha elaborado un **diagrama de clases UML**. Este diagrama refleja las principales clases del proyecto, sus atributos y métodos más relevantes, así como las relaciones de composición utilizadas para estructurar el análisis de forma modular.

8.A.2 Estructura del diagrama

El sistema está compuesto por las siguientes clases:

- **DataLoader**
 - Métodos: `load_and_merge_data()`
- **DataProcessor**
 - Atributos: `csv_path`
 - Métodos: `clean_data()`
- **TarifaSimulador**
 - Atributos: `df`
 - Métodos: `asignar_tarifas()`
- **Visualizer** (*módulo funcional, contiene varias funciones*)
 - Funciones: `generar_graficos_principales()`, `plot_ingresos_por_tarifa()`, etc.
- **Excepciones personalizadas**
 - `ClienteInvalidoError`
 - `PrecioInvalidoError`

Además, el script `main.py` actúa como **coordinador**, instanciando y utilizando estas clases para ejecutar el flujo completo del análisis.

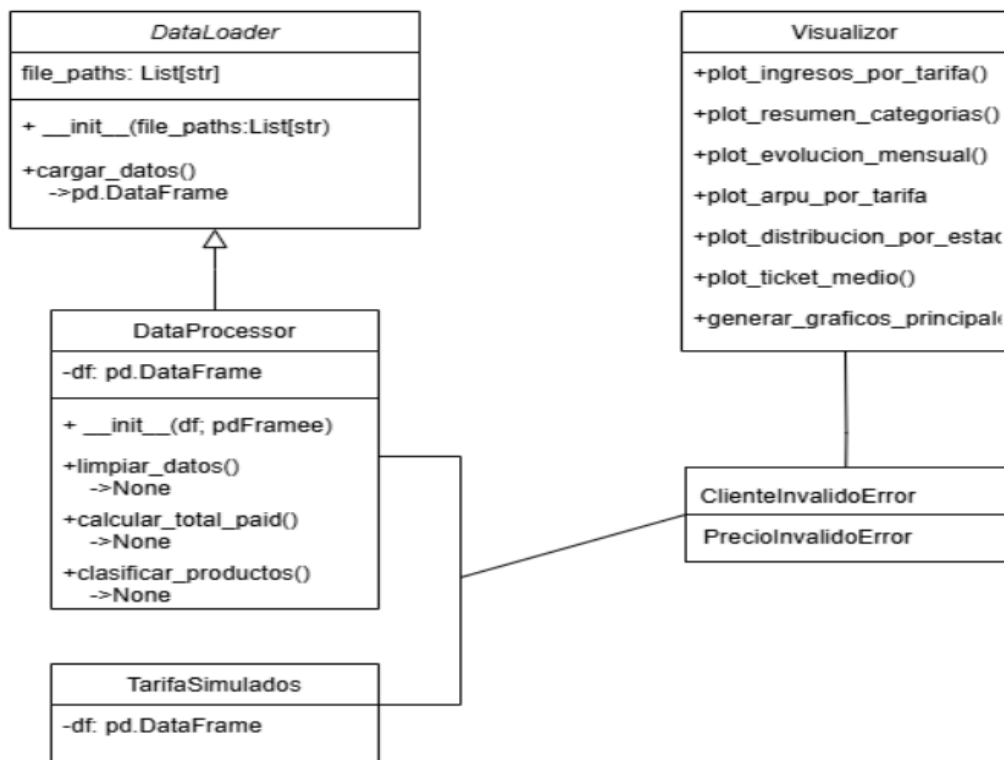
8.A.3 Relación entre clases

El patrón de diseño seguido es la **composición**:

- `main.py` **compone** las clases `DataLoader`, `DataProcessor`, `TarifaSimulador` y llama a las funciones del módulo `Visualizer`.
- Las clases no heredan entre sí, favoreciendo la independencia y reutilización.

8.A.4 Diagrama visual

Se ha incluido en los anexos un diagrama UML como imagen con el diseño explicado anteriormente:



8.B Comando de empaquetado

`python setup.py bdist_wheel --dist-dir=outputs/dist`